

Lecture 19:
Deep Learning
Convolutional Neural Networks,
Continued
ML/AI for Engineering Applications
ENV 6932

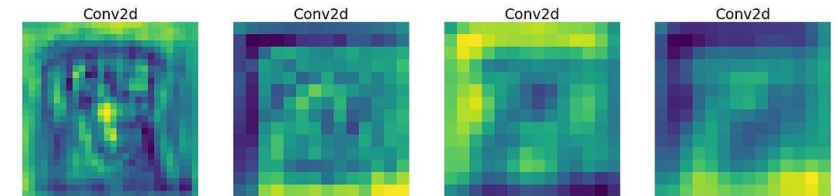
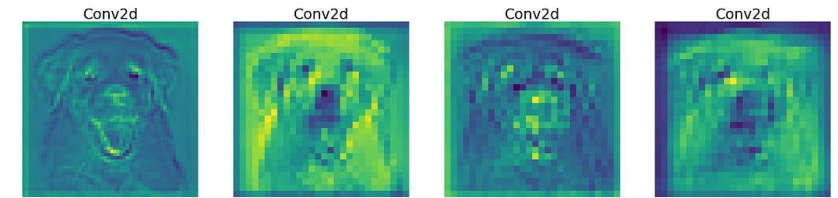
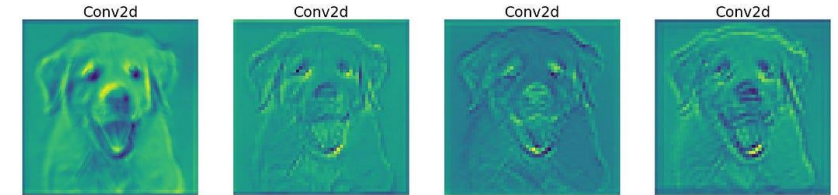
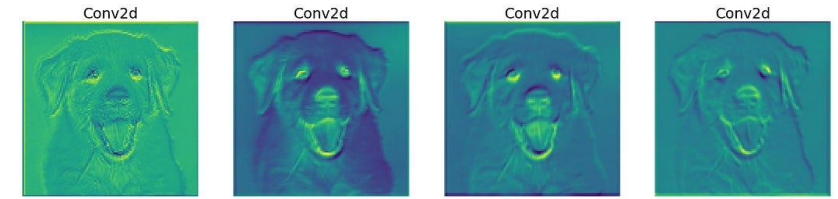
Sara Behdad
Fall 2025

Outline

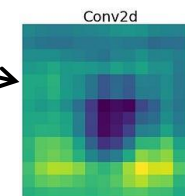
- **Recall:** Activation Functions, Loss Functions, Gradient Descent, Back Propagation, Bias-Variance in NNs
- Convolutional Neural Networks

Visualization of Feature Maps

- Learned filters in early layers look like edge detectors.
- Deeper filters: textures, shapes, object parts.
- Visualization = interpretability of CNNs.

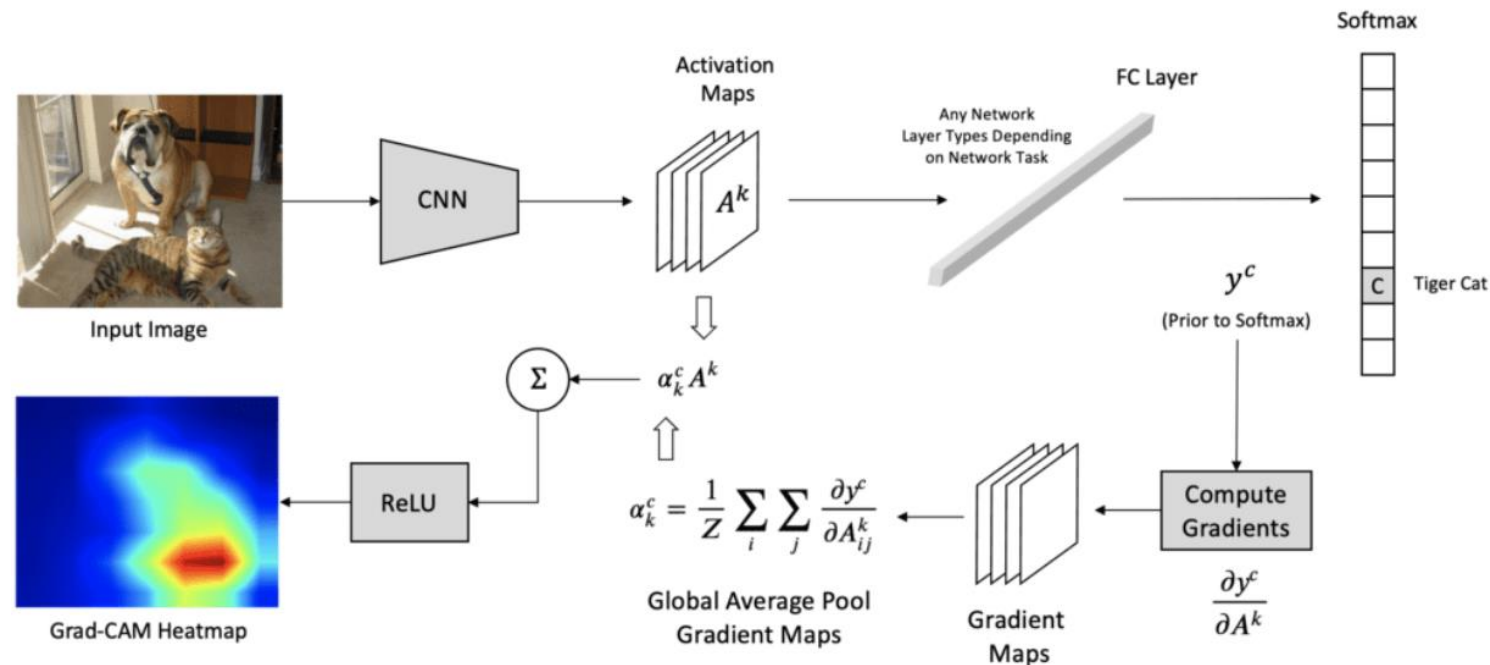


What kind of features do you think this filter detects?



Grad-CAM: Model Explainability

- Gradient-weighted Class Activation Mapping (Grad-CAM) shows regions of input that influenced model's decision.
- Helps understand what CNN 'looks at.'



How Grad-CAM works?

- Take the feature maps from the last convolutional layer.
- Compute the gradients of the target class score with respect to those feature maps.
- Weight each feature map by its gradient and therefore shows how important that map is for that class.
- Combine them into a single heatmap (overlay on the original image).

Modern CNN Architectures

Model	Major Innovation	Year
LeNet-5	Early digit recognition	1998
AlexNet	ReLU + GPU training	2012
VGG	Deep, simple	2014
GoogLeNet	Inception modules	2015
ResNet	Skip connections	2015
EfficientNet	Compound scaling	2019
Vision Transformer (ViT)	Replaces conv with attention	2020

Growth from **5 layers** to **100+ layers**

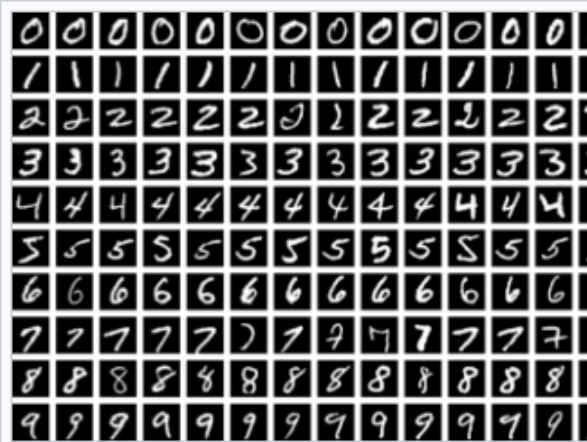
Regularization in CNNs

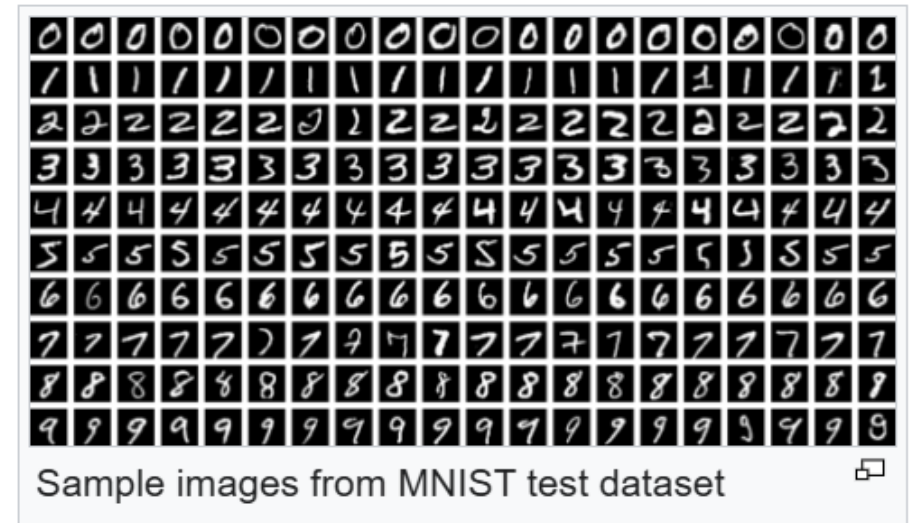
- Prevent overfitting via:
 - Dropout
 - Data augmentation
 - Weight decay (L2)
 - Batch normalization
- These make tradeoff between bias-variance and improve generalization.

Limitations of CNNs

- Require large labeled datasets.
- Poor at modeling long-range dependencies.
- Sensitive to adversarial perturbations.
- Struggle with 3D and non-Euclidean data (graphs).
- Transformers are addressing some of these limitations.

CNN Practical Implementation (Using PyTorch)

- Train a simple CNN on **MNIST** or **CIFAR-10** dataset using PyTorch.
 - Steps:
 - Define CNN architecture
 - Run forward and backward passes
 - Train and evaluate
 - Adjust hyperparameters
- 
- Sample images from MNIST test c



CIFAR-10

- The CIFAR-10 dataset consists of 60000 32x32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images.

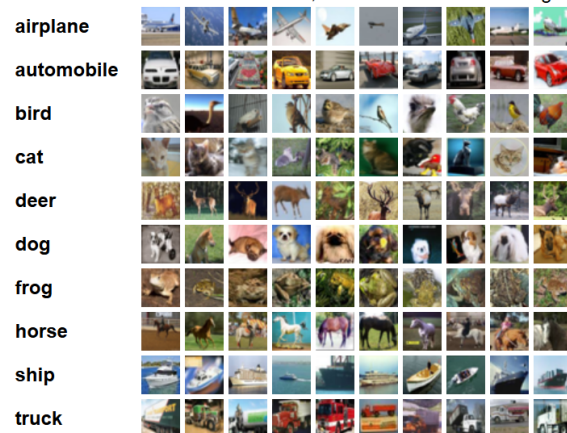
The CIFAR-10 and CIFAR-100 datasets are labeled subsets of the [80 million tiny images](#) dataset. CIFAR-10 and CIFAR-100 were created by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.

The CIFAR-10 dataset

The CIFAR-10 dataset consists of 60000 32x32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images.

The dataset is divided into five training batches and one test batch, each with 10000 images. The test batch contains exactly 1000 randomly-selected images from each class. The training batches contain the remaining images in random order, but some training batches may contain more images from one class than another. Between them, the training batches contain exactly 5000 images from each class.

Here are the classes in the dataset, as well as 10 random images from each:



The classes are completely mutually exclusive. There is no overlap between automobiles and trucks. "Automobile" includes sedans, SUVs, things of that sort. "Truck" includes only big trucks. Neither includes pickup trucks.

Example CNN Architecture

```
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.fc1 = nn.Linear(32 * 8 * 8, 10) # CIFAR-10 -> 10 classes

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # Conv1 → ReLU → Pool
        x = self.pool(F.relu(self.conv2(x))) # Conv2 → ReLU → Pool
        x = x.view(-1, 32 * 8 * 8) # Flatten
        x = self.fc1(x) # Fully connected
        return x
```

Loading Data (CIFAR-10 Example)

```
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, ), (0.5, ))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=32, shuffle=True)
```

Forward Pass and Loss

```
import torch.optim as optim

model = SimpleCNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Example batch
images, labels = next(iter(trainloader))
outputs = model(images)
loss = criterion(outputs, labels)
```

Backpropagation and Optimization

```
optimizer.zero_grad()    # 1. Clear old gradients  
loss.backward()          # 2. Backpropagate errors  
optimizer.step()         # 3. Update parameters
```

Training Loop

```
for epoch in range(5):  # 5 epochs
    running_loss = 0.0
    for i, (inputs, labels) in enumerate(trainloader):
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {running_loss/len(trainloader):.3f}")
```

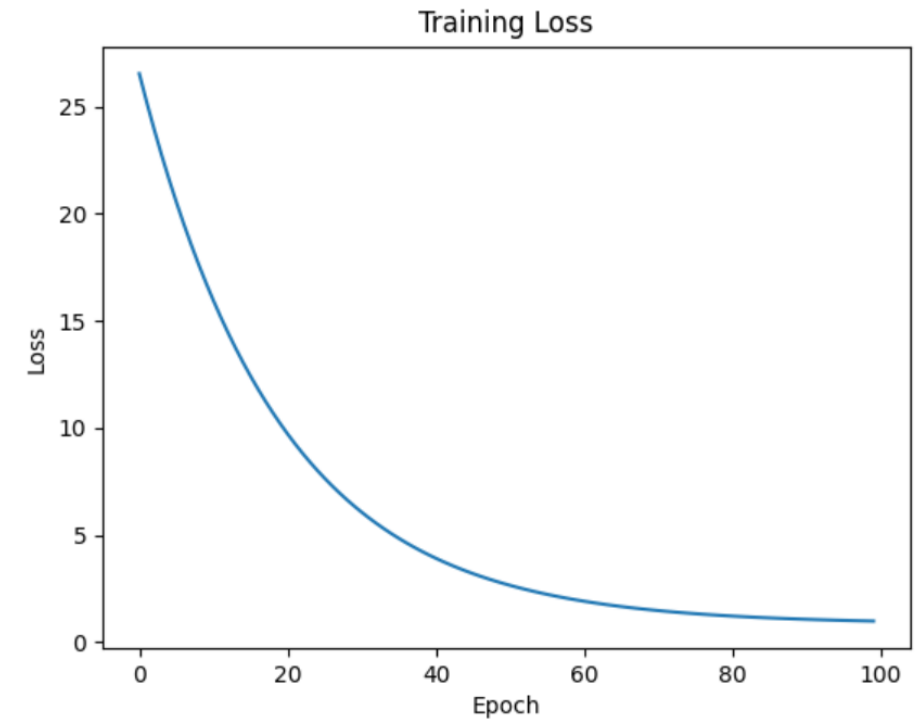
Evaluating Model Performance

```
correct, total = 0, 0
with torch.no_grad():
    for images, labels in testloader:
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Accuracy: {100 * correct / total:.2f}%")
```


Visualizing Training Progress

```
plt.plot(train_losses, label='Train')  
plt.plot(val_losses, label='Validation')  
plt.legend(); plt.xlabel('Epoch'); plt.ylabel('Loss')
```



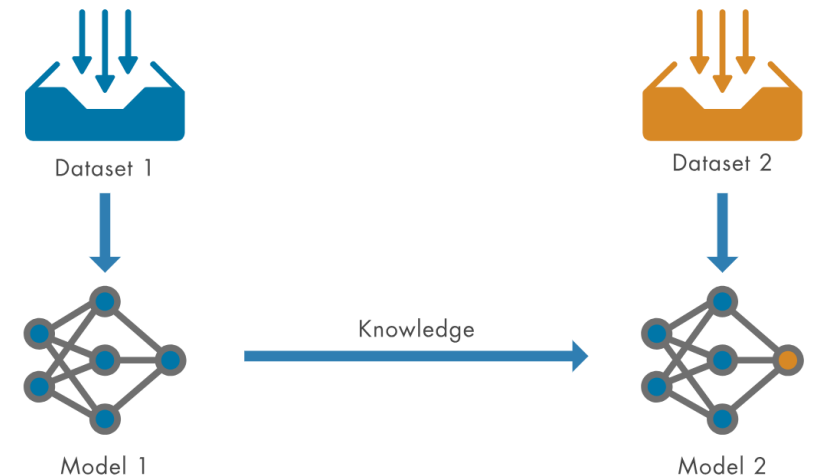
Visualizing loss at each epoch using matplotlib.

Transfer Learning and Fine-Tuning in CNNs

- *Why Transfer Learning?*
- Training a CNN **from scratch** requires:
 - Millions of labeled images
 - Weeks of training
- Most models learn **generic low-level features** (edges, textures, shapes)
- Reuse existing “knowledge” from pretrained networks therefore, saves **time, data, and compute**

What Is Transfer Learning?

- Transfer Learning = Reusing a pretrained model on a new, related task.
- CNN filters trained on one dataset capture **universal visual patterns** that generalize.
- **Mathematically:**
Given pretrained model f_{θ_0} on dataset D_0 ,
adapt parameters $\theta_0 \rightarrow \theta^*$ for new dataset D_1



Types of Transfer Learning

Approach	What We Do	When to Use
Feature Extraction	Freeze all conv layers, only train new classifier on top	Small dataset, similar domain
Fine-Tuning	Unfreeze part (last few) of conv layers, retrain with small LR	Medium dataset, moderately different domain
Full Retraining	Reinitialize and train all layers	Large dataset, very different domain

CNN as Feature Extractor

Steps:

- Take pretrained CNN (e.g., ResNet50 trained on ImageNet).
- Remove final FC layer (1000-class).
- Feed your images and then collect outputs from penultimate layer.
- Train a new classifier (e.g., Logistic Regression, SVM, or new FC).

Example:

- Input: $224 \times 224 \times 3$ image
- ResNet feature output: 2048-D vector
- New FC layer: 2048 to 4(for 4 classes)

Fine-Tuning Procedure

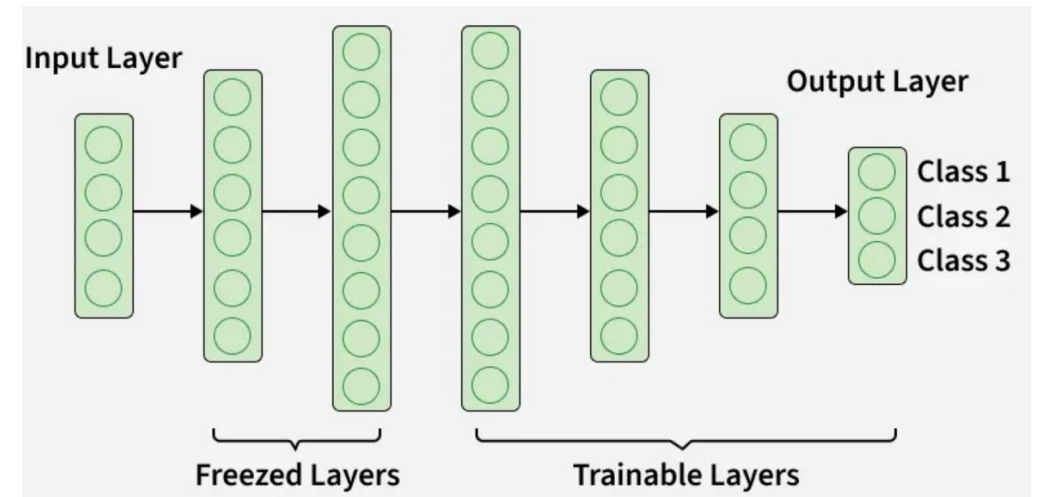
- Adjust high-level features slightly for new domain.

Steps:

- Load pretrained CNN
- Replace classifier head
- Freeze early layers (low-level features)
- Train last few conv blocks with small learning rate (1e-4)

Equation: $\theta' = \theta - \eta \nabla_{\theta} \mathcal{L}(f_{\theta'}(x)y)$

- but gradients only update unfrozen layers.



<https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-transfer-learning/>

Why Fine-Tuning Works

Layer Type	What It Learns	Transferability
Early conv	Edges, corners, colors	Reusable
Mid conv	Textures, motifs	Partially reusable
Deep conv	Object parts, semantics	Task-specific

Example: ResNet on Custom Dataset

- **Scenario:** Classify 4 defect types (crack, rust, dent, paint).

- **Steps in PyTorch:**

```
model = torchvision.models.resnet50(pretrained=True)
for param in model.parameters():
    param.requires_grad = False # freeze layers
model.fc = nn.Linear(2048, 4) # new output layer
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

- **Fine-tuning:** unfreeze last block:

```
for param in model.layer4.parameters():
    param.requires_grad = True
```


When to Use Transfer Learning

Condition	Recommended Strategy
Small dataset, similar domain	Feature extraction only
Medium dataset, slightly different domain	Fine-tune upper layers
Large dataset, very different domain	Train from scratch

Limitations of Transfer Learning

- Possible overfitting if fine-tuned on too few samples
- Transfer bias: model may inherit bias from original dataset (e.g., ImageNet)

Transfer Learning



Pros

- ✓ Faster Training
- ✓ Higher Accuracy
- ✓ Less Data Required
- ✓ Lower Computational Resources

Cons

- ✗ Limited Flexibility
- ✗ Overfitting
- ✗ Limited Generalization



CNN Variants and Advanced Architectures

- Classic CNNs (e.g. LeNet, VGG, ResNet) handle 2D spatial data (images).
- But real-world data can be temporal, volumetric, or pixel-dense.
- Modern CNNs adapt convolution operations to match data dimensionality and task type.

Data Type	CNN Variant	Example Task
1D	Temporal CNN	ECG signal classification
2D	Standard CNN	Image recognition
3D	Spatio-temporal CNN	Video action recognition
2D + dilation	Dilated CNN	Semantic segmentation
Encoder–decoder	U-Net / FCN	Biomedical imaging

From 2D to Other CNN Types

- **2D CNN:**
 - Filters slide in **height** × **width**.
 - Typical use: static images.
- If data changes type , then adapt convolutional dimensions:
- 1D CNN --- slides over *time or sequence*
- 3D CNN --- slides over *height* × *width* × *time (or depth)*

1D CNNs: For Sequential / Temporal Data

- 1D convolution slides a kernel over one dimension (e.g., time or position).
- **Input shape:** [batch_size, channels, sequence_length]
Operation: $\text{Output}(t) = \sum_{i=0}^{k-1} w_i \cdot x_{t+i}$
- **Applications:**
 - Vibration and sensor data (fault detection)
 - Speech and audio recognition
 - Financial time-series
 - Biomedical signals (EEG, ECG)

```
nn.Conv1d(in_channels=1, out_channels=32, kernel_size=3, stride=1)
```

Why 1D CNNs Work for Time Series

- CNNs can detect **local temporal patterns** (e.g., spikes, peaks).
- Learn filters such as 'shape detectors' for time windows.
- More efficient than RNNs for short/medium sequences.
- **Example:**
- Predict bearing wear using vibration data ---- filters detect repeating oscillation patterns.

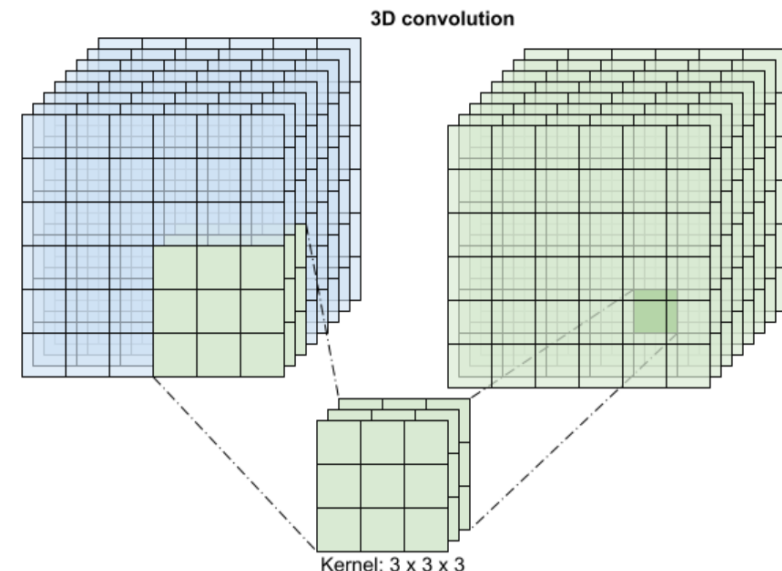
3D CNNs: For Spatio-Temporal Data

- 2D CNNs process single frames, but videos have **spatial + temporal** dimensions.
- **Input shape:** [batch, channels, depth (frames), height, width]
Operation: 3D convolution kernel slides over all three axes.
- **Applications:**
 - Video classification (e.g., running, swimming)
 - Medical imaging (CT/MRI volumes)
 - 3D object recognition in robotics

```
nn.Conv3d(in_channels=3, out_channels=64, kernel_size=(3,3,3), stride=1)
```

3D CNN Example

- **Example:**
Input = 16 video frames of 64×64 pixels each
Filter = 3×3×3 (captures spatial + temporal pattern)
- Output detects motion features such as direction, gesture, or flow.



Dilated (Atrous) CNNs

Dilated (Atrous) CNNs

- Insert spaces between kernel elements to enlarge receptive field *without increasing parameters*.
- **Dilation rate (d):**
 - Normal conv: $d = 1$
 - Dilated conv: $d > 1$ (skip pixels)
- **Formula for receptive field:** $k_{effective} = k + (k - 1)(d - 1)$
- **Applications:**
 - Semantic segmentation: capture context without downsampling
 - Audio modeling (WaveNet)